

Towards Behavior-Driven Graphical User Interface Testing

Hendrik Bänder
itemis AG
Bonn, Germany
buender@itemis.de

Herbert Kuchen
ERCIS, University of Münster
Münster, Germany
kuchen@uni-muenster.de

ABSTRACT

The majority of users interacts with an application through its graphical user interface (GUI). To ensure high quality and expected behavior, those graphical user interfaces have to be tested thoroughly. Yet, creating graphical user interface test cases is considered expensive in comparison to unit or integration tests. In addition, test cases are perceived to be expensive to run and brittle, therefore causing a lot of false negative test results. Behavior-driven test case design addresses this challenges by bringing requirement specifications and test cases closer together. Although industry-proven tools map test specifications automatically, test methods making test scripts executable need to be implemented manually. The specification language Slang introduced by this paper generates automatically executable test cases from BDD-like feature descriptions that integrate low-fidelity prototypes in form of wireframesketcher models. To quantify the economic advantage of our approach an AB/BA crossover designed experiment was conducted. The experiment showed that creating automatically executable test cases utilizing Slang takes 63% less time compared to the industry-proven tool JBehave. In addition to presenting the experiment's results, the paper elaborates on first experience from applying the approach in a large Swiss bank. The findings of our experiments are supported by results from applying our approach in real-world scenarios. In addition, experiment as well as case study participants appreciated the sophisticated editor support of Slang.

CCS Concepts

• **Computing methodologies** → **Modeling methodologies**; • **Software and its engineering** → **Domain specific languages**; **Application specific development environments**; • **General and reference** → *Empirical studies*;

Keywords

Domain-specific language, Model-Driven Engineering, GUI Test Case Generation, Behavior-Driven Development, Case Study

Copyright is held by the authors. This work is based on an earlier work: SAC'19 Proceedings of the 2019 ACM Symposium on Applied Computing, Copyright 2019 ACM 978-1-4503-5933-7. <http://dx.doi.org/10.1145/3297280.3297450>

1. INTRODUCTION

Testing of graphical user interfaces is important to verify the correct behavior of an application. Since the majority of users of an application interacts through the GUI, it requires thorough testing. To eliminate human error, graphical user interfaces should be tested by automated, reproducible test cases. Yet, automated GUI test cases are claimed to be expensive to write, time-consuming to execute, and brittle [18]. By integrating low-fidelity prototypes [7] with BDD-like requirement specifications, the introduced approach challenges the first remark.

Evolving from test-driven development, BDD has been widely adopted in agile projects in research and practice [5]. A so-called ubiquitous language as introduced in the context of behavior-driven development [10] brings requirement specifications and the respective test cases closer together. Business analysts, developers, and testers collectively describe the behavior as well as the expected business value of each feature. In the next step, features are further detailed by scenario descriptions that are automatically transformed into test methods by tools such as Cucumber [36] or JBehave [15]. To make the feature and scenario descriptions executable as test cases the generated test methods need to be implemented manually [36].

While test specifications and the underlying requirement definitions are brought together by behavior-driven development approaches, there are more artifacts in a typical development process that are not leveraged. Around 68% of agile project adopt low-fidelity prototypes [13]. The rough, conceptual sketch of a graphical user interface is used by business analysts and developers to discuss user interface requirements and potential solutions. While wireframes [4] are a typical format for low-fidelity GUI descriptions, more detailed alternatives are UI-mockups or prototypes [7].

Taking into account that in a typical development process, wireframes are available, existing BDD tools have two main shortcomings. First, they do not offer an integration, thereby, neglecting benefits when specifying BDD-like graphical user interface test cases. Second, due to the lack of integration between wireframe and BDD-like feature description, test methods have to be implemented manually. The present paper introduces a model-driven approach extending the behavior-driven development method to include low-fidelity prototypes. Thereby, the problems mentioned above can be mitigated. We present the specification language Slang that

integrates BDD-like feature specifications with wireframe mockups. Due to the integration, automatically executable test cases can be created more efficiently by fully generating the required test code.

The potential of integrating BDD-like feature descriptions with low-fidelity prototypes to fully generate automatically executable test cases is investigated in detail. A controlled experiment was conducted, to quantify the economic advantage of our approach. A AB/BA crossover experiment [20] challenges the null-hypothesis claiming that the efficiency (measured in time per test case) for specifying automated GUI test cases is equal to the efficiency of JBehave. The rationale behind choosing JBehave over Cucumber or Squish as tool for comparison is explained in Section 5.1. Further, the paper reports on first industrial experience collected at a large Swiss bank, where three differently sized components from a practical project used Slang to specify and automate graphical user interface test cases.

After discussing related work in Section 2, Section 3 exemplifies the capabilities of Slang by describing features for a calendar web application and turning them into automatically executable test cases. After describing the technical foundations in Section 4, Section 5 reports on an experiment carried out to analyze the efficiency of Slang compared to JBehave. In Section 6, we present the findings from a case study conducted at a large Swiss bank. Subsequently, Section 7 discusses the experimental results and the findings from a real-world application before we conclude in Section 8.

2. RELATED WORK

Behavior-driven development as introduced by Dan North [8] consists of six basic principles. These principles include the ubiquitous language needed to define features and scenarios as well as the automated acceptance testing with mapping rules [5]. However, generating the implementations of the test methods is not supported by any of the reviewed tools so that it remains a manual task.

In contrast, Lazăr et al. introduced an approach integrating BDD and model-driven development to enable the generation of test methods [14]. Based on executable UML enhanced by stereotypes, test methods can be generated and the BDD-like definitions become automatically executable. The focus is on "algorithmic and data-intensive types of programs" [14], while the focus of this research is on information systems. Consequently, the approach introduced by Lazăr et al. does not cover the integration of graphical user interface descriptions. In addition, the concrete textual syntax used to specify scenarios also includes business and verification code leading to complex scenario descriptions. In contrast, the specification language Slang, introduced by this paper, and especially the Feature DSL is completely independent from technical details. Further, it comes with language constructs that read like grammatically correct English sentences, which are suitable also for non-technical domain experts.

Another way to interpret sentences in natural language is natural language processing (NLP) that focuses on turning textual documents written in natural language into formal representations as explained by Collobert and Weston [6]. While the usage of NLP for behavior-driven development as

suggested by Soeken et al. [28] has no restrictions on the keywords or grammar constructs used, the ubiquitous language used by BDD tools and frameworks offers a restricted number of keywords to enable the easy interpretation of statements. Consequently an alternative solution is keyword-driven testing that describes test cases by concatenating keywords and parameters [38]. Although the behavior-driven GUI testing approach limits the number of usable keywords, the main goal is to support feature and scenario definitions that read like natural language. Further, the presented approach unifies requirement descriptions and test scripts in a single feature definition file. To summarize, the approach presented in this paper is less flexible and complex than natural language processing, because it limits the number of valid keywords such as "given", "when", "then". Yet, it is more flexible and readable than keyword-driven testing since test cases can be expressed in human-readable sentences.

The potential benefits of including GUI descriptions such as wireframes or prototypes in a model-driven development process have been elaborated by Rivero [23]. In particular Sanchez et al. [24] have shown that complete graphical user interfaces can be generated from wireframe models. An approach to combine test-driven and model-driven development was introduced by Luna et. al [17]. By deriving application models from test scripts, they leverage the benefits of both approaches such as short-development cycles and less error-prone creation of code.

Although it might be possible to generate the GUI or even the whole application from the requirements and wireframe descriptions, the present paper focuses on generating GUI test cases. There is already a variety of tools available to create automated tests for graphical user interfaces. In particular, Squish is a test automation tool that supports BDD-like test scripts [29]. However, like JBehave it will only generate a method stub that needs to be implemented manually.

Besides the tools mentioned above, there are model-based testing (MBT) approaches. As described by Utting and Legear [31], MBT focuses on creating an abstract description of the system under test in an early stage. The model is then used as input for fully generating black box system tests. The GUITAR tool introduced by Nguyen et. al [21] is an example of a model-based GUI testing tool. Models are created by a reverse engineering mechanism. These models are the basis for test oracles, system verification, and test case generation for multiple platforms. While MBT focuses on creating a test model for the system, the introduced approach uses model-driven engineering techniques to generate test cases. Following the idea of behavior-driven development, our approach merges the system definition with the test case specification.

3. MATERIAL - BEHAVIOR-DRIVEN GUI TESTING

3.1 Specifying the Calendar Web Application

The capabilities of Slang are demonstrated by a calendar web application including the corresponding wireframe, feature and mapping definitions. After a successful login, the

application offers an editable list of appointments. The web application's user interface consists of a login and an overview screen, which are modeled using a sketching tool such as Wireframesketcher [35]. The behavior and the business value of each feature is described in a Slang feature definition file. In order to connect the wireframe model elements, such as screens and widgets, to their implementations, a Slang mapping description file is created. Finally, all three artifacts are processed by a generator that creates automatically executable test scripts.

3.2 Calendar Web Application Screen Definition

Figure 1 shows a wireframe model of the login screen that has two text input fields and one button.

Browser

http://www.mycalendar.org

Username johnDoe

Password *****

Login

Figure 1: Wireframe model describing the Login screen.

After entering a valid "username" and a valid "password", the login can be executed by clicking the *Login* button. This leads to the overview screen depicted in Figure 2. The expected behavior of the login screen is described formally within a feature definition file that contains different scenarios.

3.3 Slang Feature Definition

Listing 1 shows the Slang feature specification for the login screen. It is divided into three main parts. The first part defines the *feature name*, which in this case is *Login*. The second part of the feature definition contains the *user story* that specifies the business value of the feature from the user perspective [5]. The third part starting with the keyword *Scenario* describes different scenarios for each feature using grammar constructs that are a subset of the ubiquitous language gherkin [36] enhanced by the functionality to reference wireframe model elements.

```
Feature Login

As a "Calendar User"
I want to "login to the calendar application"
In order to "manage my appointments"

Scenario ValidLogin
Given I am on the Login screen
when I type "johnDoe" in the Username textfield
and I type "Password" in the Password textfield
and I click the Login button
then I am on the Overview screen
and the Appointments table is visible
```

Listing 1: Slang Feature Description: Login Screen.

The scenario in Listing 1 displays the description of a success-

ful login. The *ValidLogin* scenario starts with the context area, which sets a specified screen into focus. In each *command section*, the first command starts with the keyword sequence **when I**, while all following commands in the same section begin with **and I**.

The first command types the value "johnDoe" into the *Username* textfield. The other two commands are concerned with entering the correct password before the *Login* button is pressed. Based on the context, the possible widgets, actions and their parameters are determined dynamically and proposed to the user by the IDE of the Feature DSL while typing. The references can be validated at any time leading to early detection of scenarios affected by renaming or removing widgets. Consequently, potential errors within a test case due to changing GUI mockups can be determined, before the test case has been executed.

Browser

http://www.mycalendar.org/overview

John Does Calendar +

Date	Start	End	Topic	Action
2016/04/01	08:00	10:00	Status Meeting	
2016/04/01	12:00	13:00	Lunch	
2016/04/04	09:00	10:00	Team Meeting	

Figure 2: Wireframe model describing the Overview screen.

The expected response of the application to the event triggered by clicking the button is described in the *assertion section*, in which the screen content is checked. The keyword sequence **then I** marks the beginning of the assertion section. The first Slang assertion within the scenario description as shown by Listing 1 asserts whether the screen has been changed to the *Overview* screen. The next assertion from Listing 1 introduced by the keyword sequence **and the** is now executed in the context of the *Overview* screen. Thus, only widgets from the *Overview* screen are valid in this part of the assertion section.

Figure 2 shows the *Overview* screen that is shown after a successful login. In order to describe further scenarios for this *Overview* screen, a separate feature description file needs to be created.

3.4 Slang Mapping Definition

After having created screen and feature descriptions, a *Slang mapping definition* to map wireframe elements to their implementation is required. The mapping file provides the information necessary to identify the implemented widgets within the application.

Listing 2 shows the Slang mapping description for the Login screen. The first part of the file states the relative URL *#/login* needed to find the screen within the application. The locator statement can change for different user interface frameworks, e.g., a relative URL for web application or a menu path for desktop applications. After the keyword

`label`, the label of the application window is displayed, used to identify and verify whether the correct screen is shown.

```
Mapping Login
locator {fragment: "#/login"}
label {"Login"}

Username{
locator: "#Username"
}
Password{
locator: "#Password"
}
Login{
locator: "#Login"
}
```

Listing 2: Slang Mapping Description: Login Screen.

The second part of the Slang mapping file contains one entry for each widget on the screen. As in the Slang feature file, `Username`, `Password`, and `Login` are references to the elements of the wireframe model. For each widget, there is one `locator`, which in case of a web application could be the ID of the element in the HTML document tree [34].

3.5 Generating Automatically Executable Test Cases

Eventually, the Feature and Mapping Generator create automatically executable test scripts and *page objects* [19], respectively. Although Feature and Mapping Generator as shown by Figure 3 rely on information from the wireframesketcher model, they are independent from each other. While the Feature Generator is executed when a Slang Feature file is saved, the Mapping Generator is triggered when a Slang Mapping file is saved.

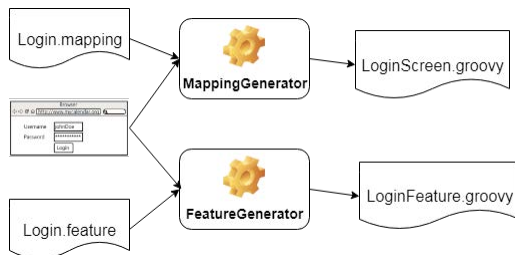


Figure 3: Generation Process.

Figure 3 shows the three files on the left, namely `Login.mapping`, `Login.screen`, and `Login.feature` that are input for Feature and Mapping Generator. The latter combines mapping and screen definition to create a *page object* per screen. The page object encapsulates all interactions, such as clicking, typing, or resolving a label, per screen.

In addition to the page objects, the Slang Feature Generator produces test scripts based on the information from the screen and feature files. Every scenario is turned into a test method containing all corresponding actions and assertions. As shown by Figure 3, the available default generators will create test scripts and page objects in the groovy programming language [12]. Test script and page object will be

explained in more detail in Section 4.5.

4. SLANG TECHNICAL FOUNDATIONS

4.1 Architecture

The language workbench to create wireframe, feature and mapping specifications is defined using the Xtext language workbench [3].

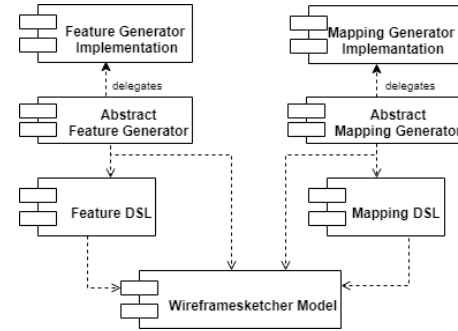


Figure 4: Slang Architecture Overview.

Figure 4 shows the architecture of Slang divided into different components. First, the Wireframesketcher component is located at the bottom of Figure 4. This component contains the graphical editor as well as the underlying formal model which is used to store screens and their widgets. The functionality offered by Wireframesketcher to create and maintain GUI models has been reused completely. However, some customizations to integrate with the Feature and Mapping DSLs as described in Subsection 4.2 have been implemented.

As shown by Figure 4 the domain-specific language parts of Slang, namely *Feature DSL* and *Mapping DSL* are located in separated and independent components. Both encapsulate the grammar definitions and the tooling implementations to support context-sensitive proposals, validations, and syntax highlighting for the Feature and Mapping DSL, respectively. Additionally, Figure 4 shows the *Abstract Feature Generator* and *Abstract Mapping Generator* components that are also independent from each other. While these components implement the integration into the Eclipse IDE in order to start the generation process when a model file has been changed, they delegate the generation to a dedicated component. By decoupling the generator implementation from the DSL definitions, generators can be switched at runtime to generate test cases for different GUI frameworks, such as web or Eclipse Rich Client Platform (RCP) [32] applications, without changing the DSLs.

4.2 Slang Wireframesketcher Integration

The GUI models are created using Wireframesketcher, a tool for creating low-fidelity prototype descriptions for GUI based applications [24]. It is well integrated in the Eclipse tool-chain and stores the GUI models as instances of an Ecore metamodel that is available in an open source repository [35]. In order to use Wireframesketcher models as basis for feature and mapping descriptions, the underlying Ecore model has been customized.

Marker interfaces have been introduced to enable context sensitive proposals within the feature definitions. Besides the existing ones such as `ClickSupport` and `SelectionSupport`, additional concepts such as `DoubleClickSupport` and `BooleanSelectionSupport` have been added. Based on the marker interfaces implemented by a widget, the proposals of available actions within the Slang editor at the current cursor position are determined. Additionally, the Resource Description Strategy [3] of the Wireframesketcher model has been extended to work with nested elements on a screen, such as widgets in containers or widgets in tables. The adjustments described above extend the capabilities of the underlying model, but have no side-effects on the remaining Wireframesketcher tooling for creating sketches.

4.3 Slang Feature DSL

In Listing 3 a slightly simplified grammar of the Slang Feature DSL in Extended Backus-Naur Form (EBNF) is provided¹. The Slang Feature DSL grammar supports all language concepts as specified by the gherkin language which is also supported by JBehave. In addition to the features used in Listing 1, the grammar from Listing 3 contains an `ImplementedCommand` and an `ImplementedAssertion` rule. These rules are a concept of the Feature DSL that can be used to specify custom behavior, e.g., setting or checking the content of a custom widget.

```
Scenario:
  'Scenario' Id
  'Given' 'I' 'am' 'on' QualifiedName 'screen'
  CodeStatement*
CodeStatement:
  ('and' 'I' | 'when' 'I') AbstractCommand |
  ('and' 'the' | 'then' 'the') AbstractAssertion |
  ('then' 'I' 'am' 'on' 'the' QualifiedName 'screen'
  )
AbstractCommand:
  Command | ImplementedCommand
Command:
  Action 'the' WidgetWrapper
Action:
  'click' | ('type' String 'in')
WidgetWrapper:
  QualifiedName ('textfield' | 'button' | 'table' |
  ...)
AbstractAssertion:
  Assertion | ImplementedAssertion
Assertion:
  WidgetWrapper 'is' ('enabled' | 'visible')
ImplementedCommand:
  String
ImplementedAssertion:
  String
QualifiedName:
  Id ('.' Id)*
```

Listing 3: Feature Grammar in EBNF (simplified; partial, ignoring e.g. rules for identifiers (Id) and strings).

Listing 3 also shows a `QualifiedName` rule used to establish the connection between *Slang feature descriptions* and *wireframe* model elements. Further, it defines a scenario as an arbitrarily mixed sequence of commands and assertions, e.g.

¹The postfix operators “?” and “*” indicate optional and arbitrarily repeated features, respectively. “|” divides alternatives. Terminal symbols appear in single quotes. For larger processes taking place on a single- or multi-screen

application.

4.4 Slang Mapping DSL

The Slang Mapping DSL is required to map the logical elements within a wireframe model such as `Screen` or `Widget` to their implementation. As shown in Listing 2, Slang mapping files are defined per `Screen` and consist of two parts.

```
Mapping:
  'Mapping' QualifiedName Url
  WidgetMapping*
Url:
  'locator' '{' ('fragment' | 'path') ':' String '}'
WidgetMapping:
  QualifiedName '{'
  'locator' ':' Id
  ('driver' ':' widgetDriver=QualifiedName)? '}'
QualifiedName:
  Id ('.' Id)*
```

Listing 4: Mapping Grammar in EBNF (partial).

The first part references the mapped screen by its `QualifiedName`, which establishes the integration between *Slang mapping definition* and *wireframe* model. Additionally, it holds a `locator` attribute containing a relative URL to find the screen within the application under test. In the example used in Listing 2, the `locator` attribute holds a fragment of a URL. For applications where screens are reachable through menus, such as desktop or mobile applications, the `locator` string is introduced by the keyword `path`.

The second part contains the Slang mapping information for all widgets on the particular `Screen`. A `WidgetMapping` consists of a reference to the wireframe model element and the `locator` attribute. The `locator` attribute is an `Id` that identifies the widget within the screen. For a web application, this might be the ID of an element in the DOM tree. In addition, the `WidgetMapping` grammar rule contains an optional `driver` attribute that references a specific implementation for retrieving widgets from the application under test. Specialized drivers can be provided to use additional properties such as label, position, or color for allocating widgets on a screen at runtime. The additional customization possibility is particularly powerful if widgets are dynamically created with an a-priori unknown ID. Finally, the reusable component feature of Wireframesketcher is supported by the mapping DSL, so that reusable screens only have to be mapped once.

4.5 Slang Feature and Mapping Generator

After having defined wireframe, feature and mapping descriptions, the Feature and Mapping generator create all artifacts required to automatically execute GUI test cases. For both generators a default implementation is available that generates test scripts for automatically testing web applications. In addition, there is a proprietary Feature and Mapping Generator for RCP applications using the Standard Widget Toolkit (SWT) [9] as GUI framework. The feature and mapping language register a builder participant in Eclipse that is called when a project is built after saving a Slang feature or mapping file. Subsequently the builder participant calls the specific generator. Based on a reference application as proposed by [33] the generator was implemented with Xtend [37],

a Java-like programming language with additional features such as type interference and template expressions. The latter are beneficial for generating source code since they facilitate working with multi-line, highly dynamic strings while maintaining formatting.

```
class LoginScreen extends Page{
    static url = "#/login"
    static at = { title == "Login" }
    static content = {
        username (required: true) { module org.bdd.module.
            TextFieldDriver, $("#username") }
        password (required: true) { module org.bdd.module.
            TextFieldDriver, $("#password") }
        submit (required: true) { module org.bdd.module.
            ButtonDriver, $("#login") }
    }
}
```

Listing 5: Generated page object.

Listing 5 shows the page object for the Login example that is generated as an instance of the **Page** class. For the login screen the generated code starts with giving the **url**. The generated code continues after the **at** attribute containing a function used to check whether the correct page is displayed. The static attribute **content** defines templates to find widgets on the inspected page. Template definitions consist of a name, additional options, a driver, and a function to find the element. The default driver for allocating widgets on a screen, e.g., **org.bdd.module.TextFieldDriver** for text fields, is used if no specific driver is specified by the mapping file. In the sample shown above, the function `$("#username")`, e.g., returns all elements on the page where the HTML DOM ID equals `#username`.

```
def "ValidLogin"() {
    given:
    to (org.specification.calendar.LoginScreen);
    when:
    username.value "johnDoe"
    and:
    password.value "Password"
    and:
    login.click
    then:
    at OverviewScreen
    and:
    waitFor { appointments }
}
```

Listing 6: Generated Feature File.

Listing 6 presents the generated test script for the Login example. The test script is generated using the groovy programming language leading to a concise, yet, executable test specification. In addition, it interacts well with the page objects also generated as groovy classes. Listing 6 starts with the **given** block that opens the login page by using the framework method **to**. The parameter of the **to** method references the Login screen page object. Based on the **url** and **title** properties (see Listing 5) it is verified that the correct screen is shown. Afterwards, the script tries to find the “username” and “password” field within the shown screen by evaluating the page objects **content** attribute (see Listing 5). Subsequently, the script sets the “username” and “password” fields value in accordance to the values stated in

the feature file. Finally, the “login” button is clicked. The **then** part contains the two assertions defined for this test script. The first one verifies whether the Overview screen is shown and the second one checks that the appointment table is shown on the screen.

5. EXPERIMENT

To verify the efficiency of defining automatically executable GUI test cases with Slang compared to JBehave, an AB/BA crossover experiment was designed and conducted.

5.1 Method

Participants were recruited by email from a Software Engineering course. Approximately one hundred students were contacted. Seventeen students volunteered to participate in the two hour experiment. Participants ranged in age from nineteen to thirty with a mean age of twenty-two. Eleven participants studied information systems, three participants studied computer science, two studied mathematics and one participant studied education. All participants had attended at least two study modules that included programming. A majority of 59 percent writes computer programs as part of study modules. Except for one participant, all volunteers had heard about automated testing and 70 percent had used automated testing at least once. The participants were allocated at random to two experimental groups with eight and nine participants, respectively.

Before the experiment started, participants completed a self-assessment questionnaire. Information was sought in relation to age, number of semesters completed and program of study. Further questions related to programming experience and self-rated programming experience in comparison to peers. Participants were asked to indicate the number of modules taken at university that included writing computer programs as well as occasions in which participants had implemented computer programs such as open source projects or as part of student work experience. Final questions asked about experience in automated testing and automated graphical user interface testing.

Following the completion of each experimental period, a tool-specific questionnaire was administered to all participants. The questionnaire used a five item Likert scale for the first four questions. The fifth question provided “yes”, “no”, “don’t know” as possible responses. The first question asked information about how well the experimental tasks were described. Afterwards, participants were asked to indicate how comfortably they felt specifying feature and mapping files using Slang or story files and test method implementations using JBehave. Further, the questionnaires sought information in relation to the tool support offered by Slang or JBehave. The fifth question asked participants if they would recommend Slang or JBehave to a friend.

Two hours in total were scheduled for questionnaire, training and the experiment. The first ten minute block of the training session started with explaining the experimental setting. Afterward, participants were introduced to the basic concepts of behavior-driven design. Within the next twenty minutes the JBehave tooling was presented. Besides the theoretical explanation, participants were asked to complete a sample

task structured exactly as the experimental tasks using JBehave. Participants were handed a list of available commands and assertions per widgets as additional material to ease test case creation. Subsequently, a sample solution was discussed with the participants. The second twenty minutes training block was structured exactly the same, yet, introducing Slang. Participants were encouraged to ask questions during the training period in order to be well prepared for the experimental session in which no questions were allowed.

Table 1: Experimental Design.

Group	Sequence	Period 1	Period 2
I	JS	Calendar	Movie
II	SJ	Calendar	Movie

The experiment was scheduled for the second hour which was divided in two thirty minute periods consisting of five minutes preparation, twenty minutes experiment, and five minutes evaluation. Within the five minutes preparation time of each experimental period the task descriptions were handed out to the participants. The task description explained which test cases should be specified for the different programs. The task description also explained when to save the files that were altered during the experiment. During the first experimental period, test cases for the Calendar application that support creating, deleting, editing and viewing the details of an appointment had to be specified using Slang or JBehave. In the second period a Movie database that can be searched for movies had to be considered. The editing of details had to be tested using Slang or JBehave.

Table 1 shows that in the experimental design, program and period are confounded. However, the first group used JBehave in the first period, whereas the second used Slang and vice versa in the second period. To avoid measuring errors, the participants started the Eclipse IDE on their computers for Slang or JBehave before the experimental period started. In addition to the task descriptions, the printed Wireframesketcher GUI mockups were given to the participants. Further, participants were allowed to use the list of available commands and assertions for each widget from the training session.

Each participant had 20 minutes to specify a maximum of 4 automated test cases. The time for each test case was measured and used as basis for the analysis of efficiency and technical merit of Slang and JBehave. Since both Slang and JBehave are file based it was possible to reconstruct the time taken to complete each test case based on the file history stored by the Eclipse IDE. After the experiment, the participant files created during the experiment and their history were collected. The time per test case was accumulated per participant and divided by the number of fully and correctly specified test cases to compute the average time per test case. Figure 5 shows the individual results for each participant. The experiment was executed as an off-line project. Participants were instructed to work independently and to not discuss the solution with their peers.

The integrated development environment Eclipse was used to specify the test cases with Slang or JBehave. Eclipse keeps a

file history that is updated every time a file is saved. The file history of Slang and JBehave files were evaluated to measure the individual times. To mitigate the copy threat the Eclipse installations and the Eclipse workspaces were separated. The Eclipse workspaces were prepared so that participants could start with their experimental task without doing any configuration activities. All materials and results including the questionnaires, task descriptions, Eclipse runtimes, and the raw result data are available for review online at: <https://tinyurl.com/SlangMaterialAndResults>.

A two period/two treatment crossover design was chosen as documented by Milliken [20]. The experiment tests the hypothesis that the efficiency of Slang and JBehave is equal. If this hypothesis can be statistically rejected (as it will), one approach (namely Slang) will turn out to be more efficient.

As presented in Table 1 the two groups adhere to a different sequence of applying either Slang or JBehave first. Group 1 applied JBehave in the first period followed by Slang in period two, while Group 2 applied Slang in period one followed by JBehave in period two. It is hypothesized that the sequence does not affect the measurement. Since both Slang and JBehave were introduced before the experiment and participants had the opportunity to gain practice in both tools the carryover effect should be minimal. Nevertheless, a factor for carryover was included in the design. As stated by Vegas et al [27] three variables are unavoidably confounded in AB/BA crossover designs, namely sequence, carryover, period*treatment interaction. In the present study, sequence as a factor was included in the model as discussed in Subsection 5.2. In order to control the error of measurements, participants were allocated to alternate workstations according to the sequence allocation.

The experiment was designed to compare the efficiency of specifying automatically executable test cases with Slang to the efficiency of existing tools for defining automatically executable BDD-like test cases, such as JBehave [15], Cucumber [36], or Squish [29]. Therefore, the comparison to alternative approaches for defining graphical user interface test cases such as Sikuli [26] or Silenium [25] as well as model-based testing approaches such as Guitar [21] are beyond the scope of this paper. All three BDD tools mentioned above support the gherkin language, provide sophisticated editor support as Eclipse IDE plugins, and generate test methods for implementing the scenario descriptions. Since it would have been difficult to introduce all three tools within a reasonable time frame, only one tool was chosen as representative for BDD-like test case specification. Due to the fact that Java was the only programming language all participants mastered, Squish that does not support test scripts written in Java was excluded. Although JBehave and Cucumber are equivalently suitable based on their features, JBehave was chosen as tool of comparison because of the mature Eclipse IDE integration. This is particularly evident in the code completion offered by the JBehave editor that provides not only potential keywords, but also sample templates to quickly create specifications.

Data was analyzed using a linear mixed model recommended for repeated measures in a within-subject crossover design [27]. The data was analyzed using R [22] with the test in linear mixed models package [16] that enhances the lme4 package [2] by tests such as type III F test for fixed effects.

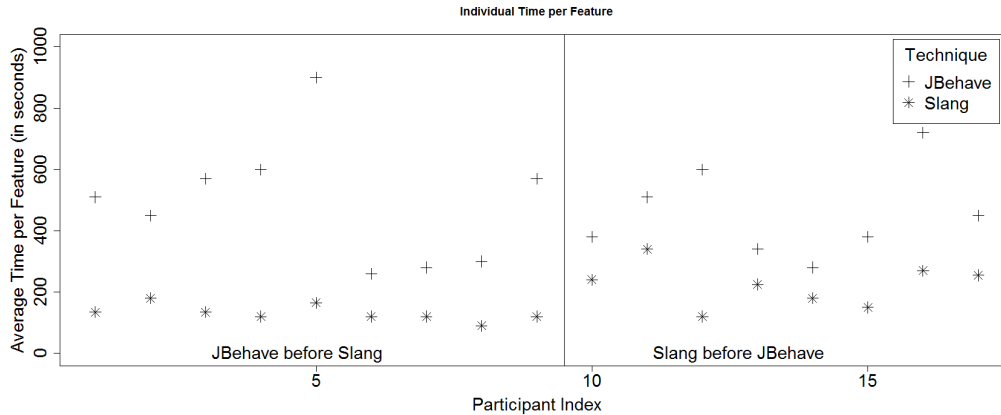


Figure 5: Individual Results Grouped by Sequence.

5.2 Results

The descriptive statistics of the experiment are shown in Table 2. As presented by Table 2 it took 174 seconds on average to specify an automated test case using Slang. In comparison, using JBehave 476 second were needed on average. The significance threshold was set at 0.05.

Table 2: Required Time for Creating Automated Test Cases (in seconds).

Technique	Mean	Std. Dev.	Std. Error.	N	Sig.
JBehave	476	174	42	17	<.001
Slang	174	69	17	17	<.001
Total	325	121		34	

Figure 5 presents the individual results sorted by sequence. The horizontal axis shows the participant index and the vertical axis shows the time it took on average to fully and correctly specify a test case. The diagram area is divided by a vertical line separating the participants that used JBehave before Slang on the left from the participants that used Slang before JBehave on the right. The points on the diagram mark the individual results per participant and technique. Figure 5 indicates that each participant regardless of the sequence in which the techniques were used achieved a more efficient result using Slang.

The linear mixed model used for the analysis includes the following fixed factors: technique (treatment), period (confounded with program), and sequence (confounded with period*technique and carryover). Participant is modeled as random factor nested within sequence.

Analyzing the data using a linear mixed model is based on the assumption that the residuals are distributed normally. In order to apply the linear mixed model, the measured data was transformed using the Box-Cox transformation [11]. The boxcoxm package by Almohaimeed and Einbeck [1] was used to find the optimal λ (0.5) for the transformation.

Table 3 gives the result of the type III F test revealing that technique is the only significant factor (sig. 1.164×10^{-6}). In

addition, the test of the remaining fixed factors as reported in Table 2 shows:

- The period factor is not significant and therefore has no effect on the results.
- Sequence is not significant as shown by Table 2.

Based on the results reported in Table 3 the null-hypothesis can be rejected.

Table 3: Type III Test of Fixed Effects.

Source	Nomina- tor df	Denomi- tor df	F	Sig.
Period	1	14.916	2.91	0.1
Technique	1	15.825	57.65	<.001
Sequence	1	22.358	0.004	0.95

The results from the tool-specific questionnaires handed to the participants after each period are shown in Figure 6. The first stacked bar graph shows that 88 percent of participants considered the experimental tasks to be explained well. The second stacked bar graph presents how comfortably participants felt when creating feature or story files using Slang or JBehave, respectively. For Slang the majority of participants stated that they felt very comfortably. For JBehave the majority answered “neutral”. The third stacked bar graphs represents the result for the question how comfortably participants felt when creating mapping files or test method implementations using Slang or JBehave, respectively. While four participants felt uncomfortably using JBehave, all participants using Slang felt at least neutral with a majority feeling “very comfortable”. The fourth stacked bar graph shows the evaluation results for the tool support offered by Slang and JBehave. While 88 percent totally agreed with the statement that they felt well supported by Slang, only one participant felt well supported by JBehave. Moreover, nearly 60 percent felt not well supported (neutral, disagree, or totally disagree) by the JBehave tooling. The fifth stacked bar graph indicates that all participants will recommend

Slang to a friend. Yet, only 20 percent of participants would recommend JBehave.

6. CASE STUDY: BANKING APPLICATION

After having tested the behavior-driven GUI testing approach with an AB/BA crossover designed experiment, first real-world project experiences have been gained within the banking sector. The application under test is used by specialized bank employees to create and maintain banking products. In contrast to the applications used in the experiment, the considered banking application was implemented as an Eclipse-based rich client platform (RCP) solution. Nevertheless, the same wireframe meta model as well as the same feature and mapping DSLs were used. This supports our claim that both DSLs are sufficiently general to cope with different UI frameworks. The generator implementation, however, had to be changed to generate automated test cases for RCP applications. SWTBot a framework specialized for testing SWT applications, was chosen [30] as tool for automating the test specifications.

Table 4: Used widgets

Project	Screens	Buttons	InputFields	Tables
SWT1	40	95	33	24
SWT2	12	10	16	10
SWT3	2	5	2	2

During the case study, three differently sized components from the application under test applied Slang to specify and generate graphical user interface test cases. Table 4 shows the number of screens and widgets implemented for each of the three SWT components. While **SWT3** is the smallest component with only two screens and a total of 9 widgets, **SWT2** contains 12 screens and more than 35 widgets. As shown by Table 4, **SWT1** consists of the largest number of screens and more than 150 widgets. Consequently, **SWT1** has the largest number of test cases as Table 5 points out. In addition, Table 4 substantiates the need for advanced table support. Further, the high number of **Buttons** and **InputFields** corroborates the widgets supported by Slang.

Table 5: Behavior-Driven UI Testing measures

Project	Number of test cases	Percentage of Generated Code
SWT1	42	94
SWT2	15	100
SWT3	4	100

Table 5 shows the number of test cases as well as the percentage of generated test case code per project. The tests verify different GUI behavior ranging from simple sorting and filtering on tables to forcing certain error dialogs. Obviously, **SWT3** offers very basic functionality that is described and verified by four fully generated test cases as indicated by Table 5. These four test cases verify that all expected widgets are shown. In **SWT2** all available screens have been checked for expected layout and in addition the page flow between the

12 screens was specified. The component **SWT1** also contains more sophisticated scenarios to create new elements in a table and to force particular error dialogs.

The percentage of generated code was determined by analyzing the manually implemented lines of code in relation to the overall number of lines of code. Table 5 shows that the test cases for **SWT2** and **SWT3** were generated completely, while **SWT1** contains 6 percent manually written test code. Since UI widgets and functionalities of **SWT2** and **SWT3** could completely be covered by the behavior-driven GUI testing approach, no manual implementation was required. In **SWT1**, however, the closing of editors in a dirty state (with unsaved changes) needed to be implemented manually.

In the three projects, Wireframesketcher as well as BDD-like feature and scenario descriptions were already used. Therefore, Slang could be utilized by business analysts and testers after a short training. Furthermore, in comparison to manually writing automated test cases, the time a developer spent decreased significantly by limiting their efforts to solely providing the mapping definitions and the manually implemented commands.

In comparison to defining requirements in natural language the introduced approach forces business analysts to abide to a rather restrictive format. Since BDD was already known by the business analysts involved, this restriction was no obstacle. In general, the feedback from business analysts and testers was especially emphasizing the advanced validation possibilities that helped to keep requirements specifications and wireframe GUI descriptions consistent. In addition, the concept of reusable components as introduced by Wireframesketcher that is supported throughout the whole Behavior-driven GUI testing approach was much appreciated. Especially in projects **SWT1** and **SWT2**, the feature was used extensively to describe complex screens.

In contrast to the existing approach, where specialists were required to implement and maintain automated tests, Slang significantly decreased the amount of manually implemented test code. Subsequently, by providing automatic creation of automated test cases less developers resources were required to make test specification executable. In addition, test specification and wireframesketcher models stayed consistent due to early validations on model-level. Further, the amount of false-negative results due to mismatches between specification and screen implementation decreased significantly. By utilizing Slang, the project team was able to provide automated test cases deduced from the scenario specifications.

7. DISCUSSION

Subsection 5.2 presents the results of our experiments showing that Slang is more efficient than JBehave and works well for users with different backgrounds. Further, the quantitative results were endorsed by the qualitative results where participants expressed that they felt comfortably using Slang.

The results from Table 3 show that “technique” is the only significant factor for the increased performance so that the null hypothesis can be rejected. Specifying fully automated test cases using Slang on average needs 63% less time than using JBehave. The main advantage of Slang is the ability

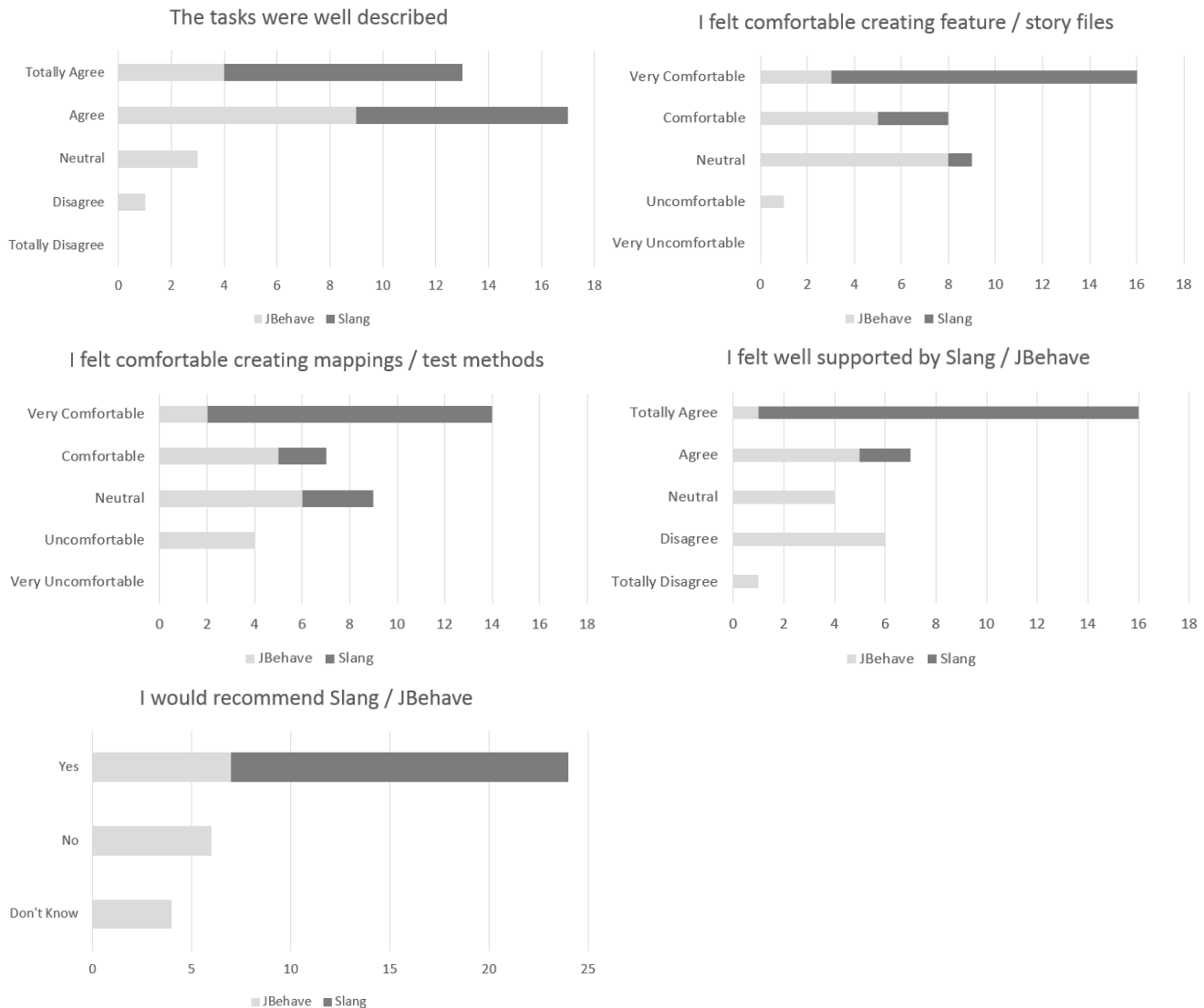


Figure 6: Participant Evaluation Results of Slang and JBehave.

to fully generate the test methods required to execute the scenarios. Leveraging, the higher degree of automation for making test cases executable is the basis for the increased efficiency.

In addition as shown in Figure 6, the participants have reported that they felt better supported by Slang. This can be explained by the mature editor support and the context sensitive integration of wireframe models that guides participants through the process of creating scenarios and mappings.

The experiment has shown that the integration of wireframe models and scenario descriptions enables the complete generation of test methods. As Lazăr et al. [14] have shown how scenario descriptions can be combined with UML artifacts to generate test methods for algorithms, we have shown that scenario descriptions combined with wireframe models can be used to generate test methods for graphical user interfaces.

Regarding one of the main disadvantages of automated graphical user interface tests, namely being expensive to write, the experimental results show that Slang decreases the time required per test case. Our approach, automates the most time consuming part of the test case creation, which is the implementation of the page objects. Thereby, Slang is more efficient than JBehave and as we believe more efficient than other behavior-driven test tools such as Cucumber or Squish since they all require a manual test-method and page-object implementation.

As explained in Section 2, our approach is less flexible and complex than natural language processing but more flexible and readable than keyword driven testing. Since test cases could be specified with Slang as well as with JBehave the additional restrictions introduced by the Feature DSL did not prevent the test case creation. Thereby, Slang allows non-technical users to specify automated test cases. Due to the fact that all participants had at least some program-

ming experience, they were able to create test cases with JBehave as well as with Slang. However, as Figure 5 reveals, creating test cases using JBehave took longer, especially, for participants self-assessing their programming skills as “unexperienced” or “very unexperienced” compared to their peers.

Threats to validity. The validity of the experimental results is a key issue. The AB/BA crossover design shown in Table 1 addresses the inherent threats to validity [27] in the following way:

Period and program are confounded in the selected design. As indicated by Table 3 period is not a significant factor in our model. Therefore, all threats to validity associated with the experimental session, namely learning by practice and tiredness/boredom, are not occurring. Since period and program are confounded, program is also not significant.

There is no threat of technique learning, because participants only apply each technique once. Thereby, any improvement due to repeated application of a technique is minimized. Moreover, there is no threat of object learning since every participant applies each technique only to one requirement specification (Calendar or Movie application). The copy threat can be ruled out, since participants use a different Eclipse installation and workspace for the requirement specification in each period.

As shown in Table 3, “sequence” is not a significant factor in the model. Therefore, carryover as a threat to validity is not present. Moreover, period $\times$ technique interaction which is also confounded with sequence is not occurring. Hence, there is no optimal sequence validity threat present. An additional threat is the fact that Slang is only compared to JBehave while there are other tools, such as Cucumber [36] or Squish [29] that may be used for behavior-driven development. However, since all tools do not support the complete generation of test methods, they are equally suitable for the experiment.

Inference. The experimental results show that Slang offers a language to efficiently specify automated graphical user interface test cases. Hence, even users with limited programming skills and non-technical experts are able to specify test cases in form of readable sentences. Slang is most relevant, effective, and efficient for projects using wireframe models. In a commercial setting, Slang would enable team members with various background to participate in creating test cases that are automated eventually by the introduced generators.

The case study conducted at a large Swiss bank supports the findings from the AB/BA experiment. In contrast to the idea of fully generating all test cases, the practical experience has shown that there are cases in which a flexible way to include manually implemented code is necessary. Overall the statistics show that the approach is capable of generating large parts of automated test cases, but there is still room for improvement. As Table 5 shows, the number of test cases in the three SWT components varies significantly. This is justified by the fact that the projects provide differently comprehensive UI functionalities.

The percentage of generated code was determined by analyzing the lines of code (LOC) manually implemented for

`ImplementedCommands` and `ImplementedAssertions` in relation to the overall lines of produced test code. Although LOC are a debatable measure for computer programs, they in this case show that the majority of code can be created automatically. In addition, the feedback from the industry partner confirmed that Slang is easy to use. Especially the editor support and the sophisticated integration with wireframe mockups was emphasized. Although the findings from the case study support results from the controlled experiment, further studies of real world projects utilizing Slang have to be conducted.

8. CONCLUSION

The introduced specification language Slang integrates BDD-like feature description with wireframe model elements. In combination with a mapping definition, automatically executable test cases can be generated fully. The three main preconceptions of GUI test cases, namely that they are expensive to write, time consuming to execute, and brittle, are rebutted or at least mitigated by our approach.

The study presented in Section 5 has clearly demonstrated that test cases can be created much more efficiently, since existing models, namely wireframe and feature model, can be integrated. Further in direct comparison to JBehave, specifying automatically executable test cases using Slang requires 63 percent less time. The experimental results have been confirmed by three projects in the financial sector that have been analyzed during a case study. For the three components, at least 96 percent of the test code could be generated. Moreover, the users in the experiment as well as in the real world project were highly satisfied by the mature editor support of Slang.

Future work may focus on comparing Slang to other behavior-driven development tools, such as Cucumber or Squish. Moreover, additional research is required to verify the expected effects of Slang on mitigating the risk of brittle test cases. Further, additional case studies should be conducted to verify the practical relevance of Slang in industry sized projects.

The contribution of this paper is threefold: first, a Specification Language (Slang) has been presented that consists of feature and mapping descriptions. Second, two generators have been developed that combine wireframe, feature and mapping definition to generate automatically executable GUI test cases. Thirdly, the experimental results clearly show the technical merit and improved efficiency of Slang not only in example projects but also in industry context. Furthermore, Slang was positively endorsed by all those participating in the experiments and in the projects in the financial sector. Which can be seen as an indication of Slang's commercial value supported by first industry application results.

9. REFERENCES

- [1] A. Almohaimeed and J. Einbeck. *boxcoxmix: Response Transformations for Random Effect and Variance Component Models*, 2017. R package version 0.15.
- [2] D. Bates, M. Mächler, B. Bolker, and S. Walker. Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1):1–48, 2015.

- [3] L. Bettini. *Implementing domain-specific languages with Xtext and Xtend*. Community experience distilled. Packt Pub, Birmingham, UK, 2013.
- [4] D. M. Brown. *Communicating design: Developing web site documentation for design and planning*. - Previous ed.: 2007. - Includes index. Voices that matter. New Riders, Berkeley, Calif., 2nd ed. edition, 2011.
- [5] C. Solis and X. Wang. A study of the characteristics of behaviour driven development. In *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications*, pages 383–387, Washington, DC, USA, 2011. IEEE Computer Society.
- [6] R. Collobert and J. Weston. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167, New York, NY, USA, 2008. ACM, ACM.
- [7] A. Coyette, S. Kieffer, and J. Vanderdonckt. *Multi-fidelity Prototyping of User Interfaces*, pages 150–164. Springer-Verlag, Berlin, Heidelberg, 2007.
- [8] Dan North. Introducing bdd, 2018.
- [9] Eclipse. Swt - the standard widget toolkit, 2018.
- [10] E. Evans. *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley, Boston, Mass. and Munich [u.a.], 20. print edition, 2014.
- [11] George E. P. Box and David R. Cox. An analysis of transformations. *Journal of the Royal Statistical Society*, 26(2):211–252, 1964.
- [12] groovy-lang.org. groovy - a multi-faceted language for the java platform, 2018.
- [13] Z. Hussain, W. Slany, and A. Holzinger. Current state of agile user-centered design: A survey. In A. Holzinger and K. Miesenberger, editors, *HCI and Usability for e-Inclusion: 5th Symposium of the Workgroup Human-Computer Interaction and Usability Engineering of the Austrian Computer Society, USAB 2009, Linz, Austria, November 9-10, 2009 Proceedings*, pages 416–427. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.
- [14] Ioan Lazăr, Simona Motogna, and Bazil Pârv. Behaviour-driven development of foundational uml components. *Electronic Notes in Theoretical Computer Science*, 264(1):91–105, 2010.
- [15] JBehave. What is jbehave, 2018.
- [16] A. Kuznetsova, P. B. Brockhoff, and R. H. B. Christensen. lmerTest package: Tests in linear mixed effects models. *Journal of Statistical Software*, 82(13):1–26, 2017.
- [17] E. R. Luna, J. Grigera, and G. Rossi. Bridging test and model-driven approaches in web engineering. In *International Conference on Web Engineering*, pages 136–150, Berlin, Heidelberg, 2009. Springer, Springer Berlin Heidelberg.
- [18] Martin Fowler. Test pyramid, 2012.
- [19] Martin Fowler. Pageobject, 2013.
- [20] G. A. Milliken and D. E. Johnson. *Analysis of messy data . Volume 1 Designed Experiments/ by George A. Milliken, Dallas E. Johnson*. CRC Press, Boca Raton, 2nd ed. edition, 2009-.
- [21] B. N. Nguyen, B. Robbins, I. Banerjee, and A. Memon. Guitar: an innovative tool for automated testing of gui-driven software. *Automated software engineering*, 21(1):65–105, 2014.
- [22] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2017.
- [23] J. M. Rivero, G. Rossi, J. Grigera, J. Burella, E. R. Luna, and S. Gordillo. *From Mockups to User Interface Models: An Extensible Model Driven Approach*, pages 13–24. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010.
- [24] O. Sanchez Ramon, J. Garcia Molina, J. Sacher Cuadrado, and J. Vanderdonckt. GUI Generation from Wireframes, 2013.
- [25] Selenium. Seleniumhq - browser automation, 2017.
- [26] Sikuli. Sikuli - automate anything you see, 2017.
- [27] Sira Vegas, Cecilia Apa, and Natalia Juristo. Crossover designs in software engineering experiments: Benefits and perils. *IEE Transactions on Software Engineering*, 42(2):120–135, 2016.
- [28] M. Soeken, R. Wille, and R. Drechsler. Assisted behavior driven development using natural language processing. In *Proceedings of the 50th International Conference on Objects, Models, Components, Patterns, TOOLS’12*, pages 269–287, Berlin, Heidelberg, 2012. Springer-Verlag.
- [29] Squish. Squish - reliable gui test automation that works, 2017.
- [30] SWTBot. What is swtbot, 2018.
- [31] M. Utting and B. Legeard. *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2007.
- [32] L. Vogel. *Eclipse rich client platform: The complete guide to Eclipse application development*. Vogella, Hamburg, third edition, 2015.
- [33] M. Völter and T. Stahl. *Model-Driven Software Development: Technology, Engineering, Management*. Wiley Series in Software Design Patterns. John Wiley & Sons, New York, NY, 2013.
- [34] W3C. Document object model (dom), 2018.
- [35] Wireframesketcher. Wireframesketcher, 2018.
- [36] M. Wynne, A. Hellesøy, S. Tooke, and J. Carter. *The Cucumber book: Behaviour-driven development for testers and developers*. The pragmatic programmers. The Pragmatic Bookshelf, trademark of The Pragmatic Programmers, LLC, Raleigh, North Carolina, second edition, 2017.
- [37] Xtend. Xtend introduction, 2018.
- [38] A. Zylberman and A. Shotten. Test language: Introduction to keyword driven testing, 2010.

ABOUT THE AUTHORS:



Hendrik Bänder received the Bachelor's degree in Information Systems from the Berufsakademie Weserbergland and the Master's degree in Information Systems from the University of Münster. Currently, he is an external Ph. D. student at the Department of Practical Computer Science at the University of Münster. His research focuses on increasing model utilization in software engineering. Hendrik works at itemis AG as IT-Consultant utilizing model-driven approaches in various industrial projects.



Herbert Kuchen is a Professor for Practical Computer Science at the University of Münster. Since 2004 he is Director of the European Research Center for Information Systems (ERCIS). His research focuses on Model-Driven Software Development, Parallel Programming, Testing, Functional Logic Programming, and E-assessment. Herbert Kuchen supervises, amongst others, research projects concerned with Model-Driven Mobile Development, Constraint-based Test Case Generation, and Logic-Imperative Languages.